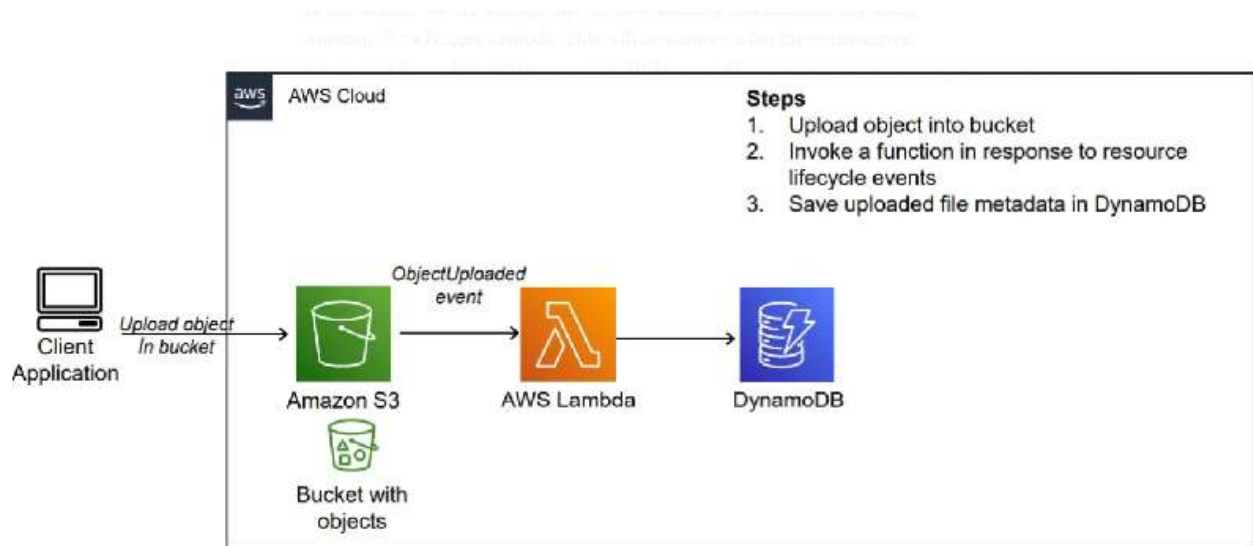


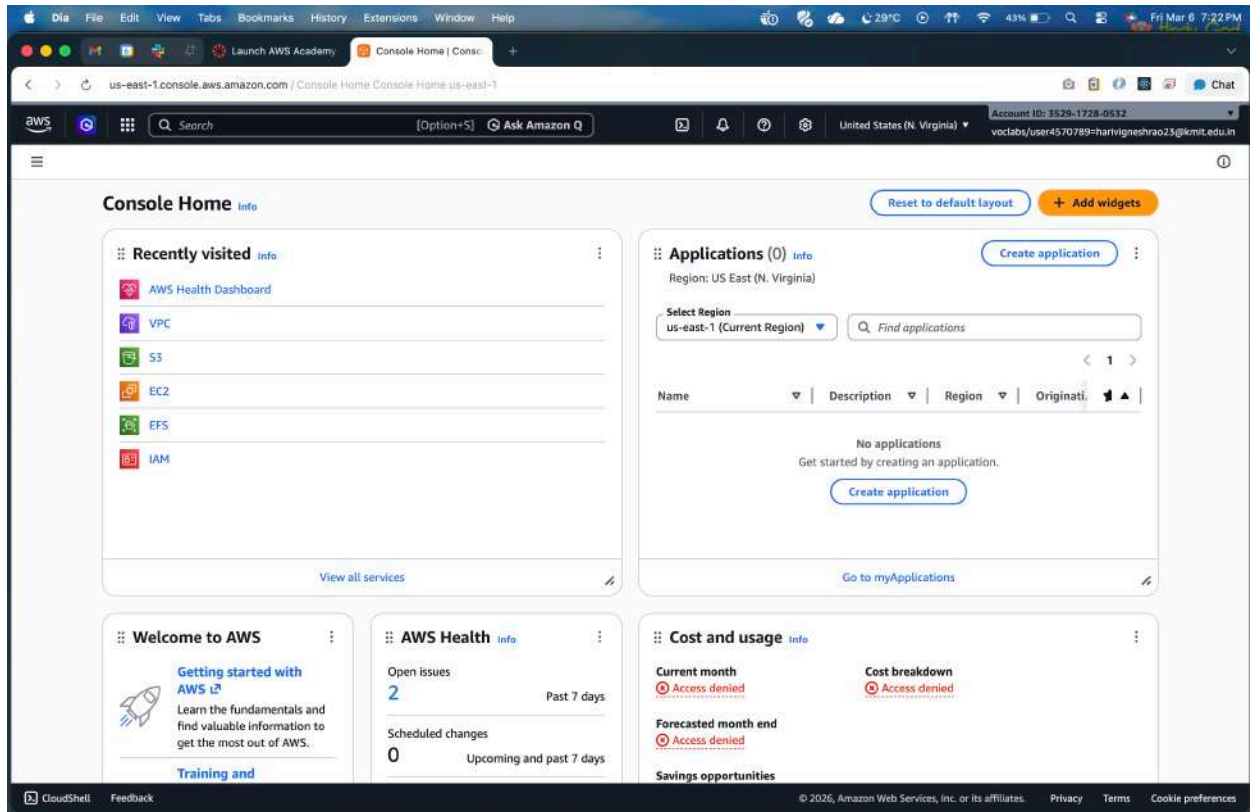
HARI VIGNESH RAO 23BD1A052Q

Setting up an AWS Lambda function to trigger whenever an object is uploaded to an S3 bucket and update a DynamoDB table

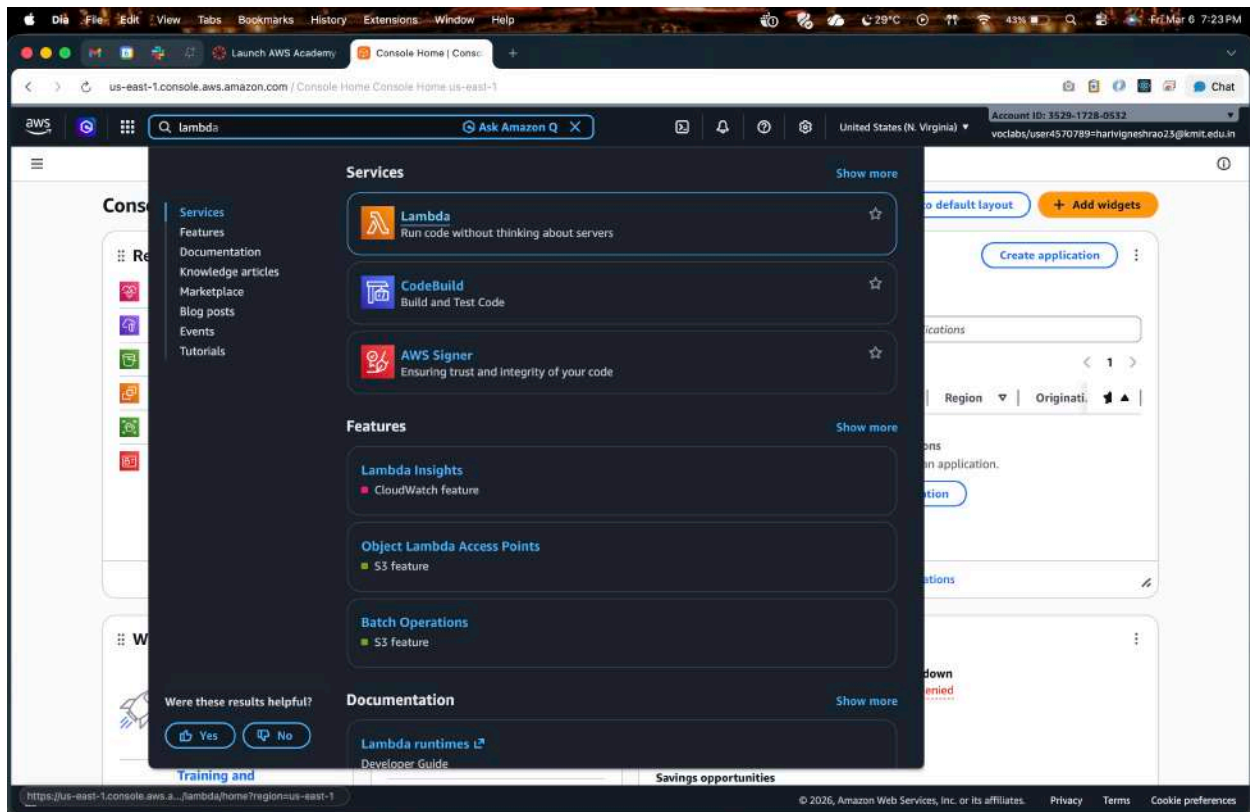


step-by-step procedure for setting up an AWS Lambda function to trigger whenever an object is uploaded to an S3 bucket and update a DynamoDB table:

1. Create an AWS Account: If you haven't already, sign up for an AWS account at <https://aws.amazon.com/>.
2. Access AWS Management Console: Log in to the AWS Management Console using your AWS account credentials.

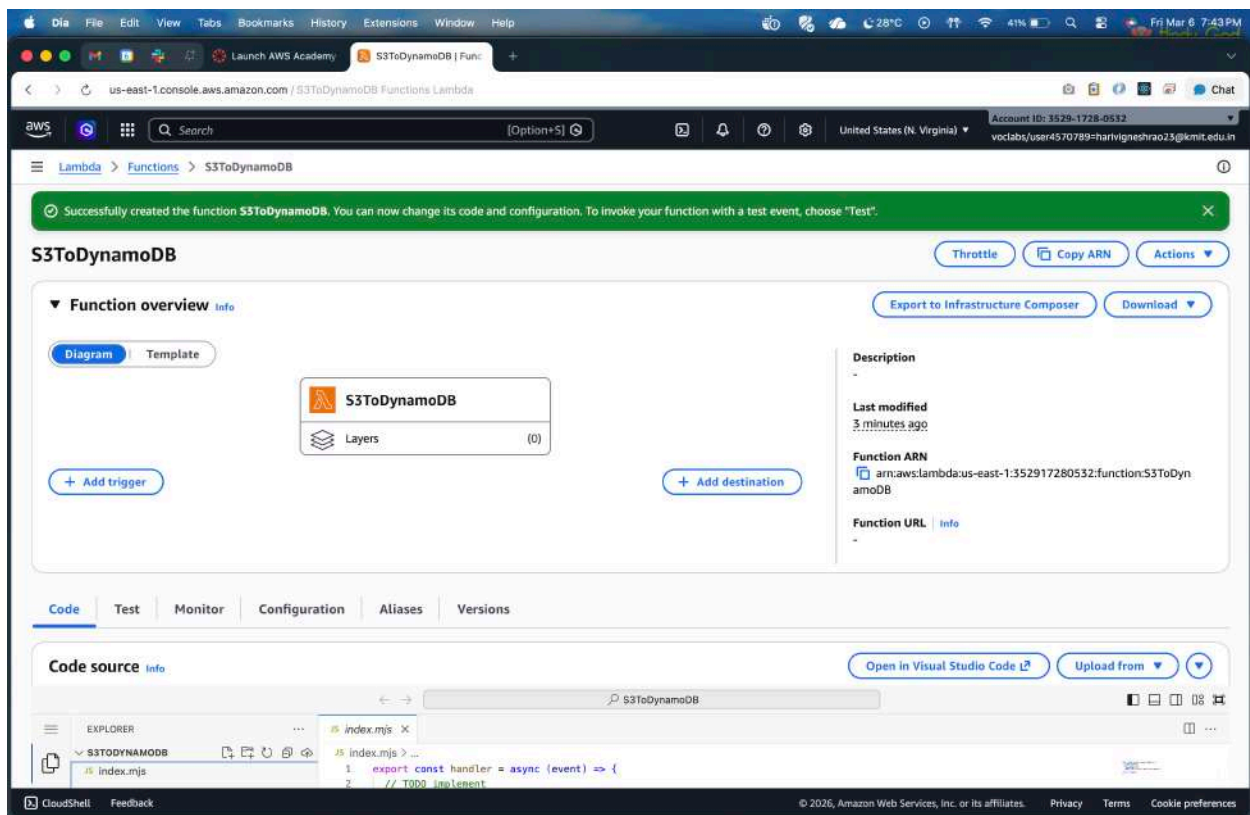
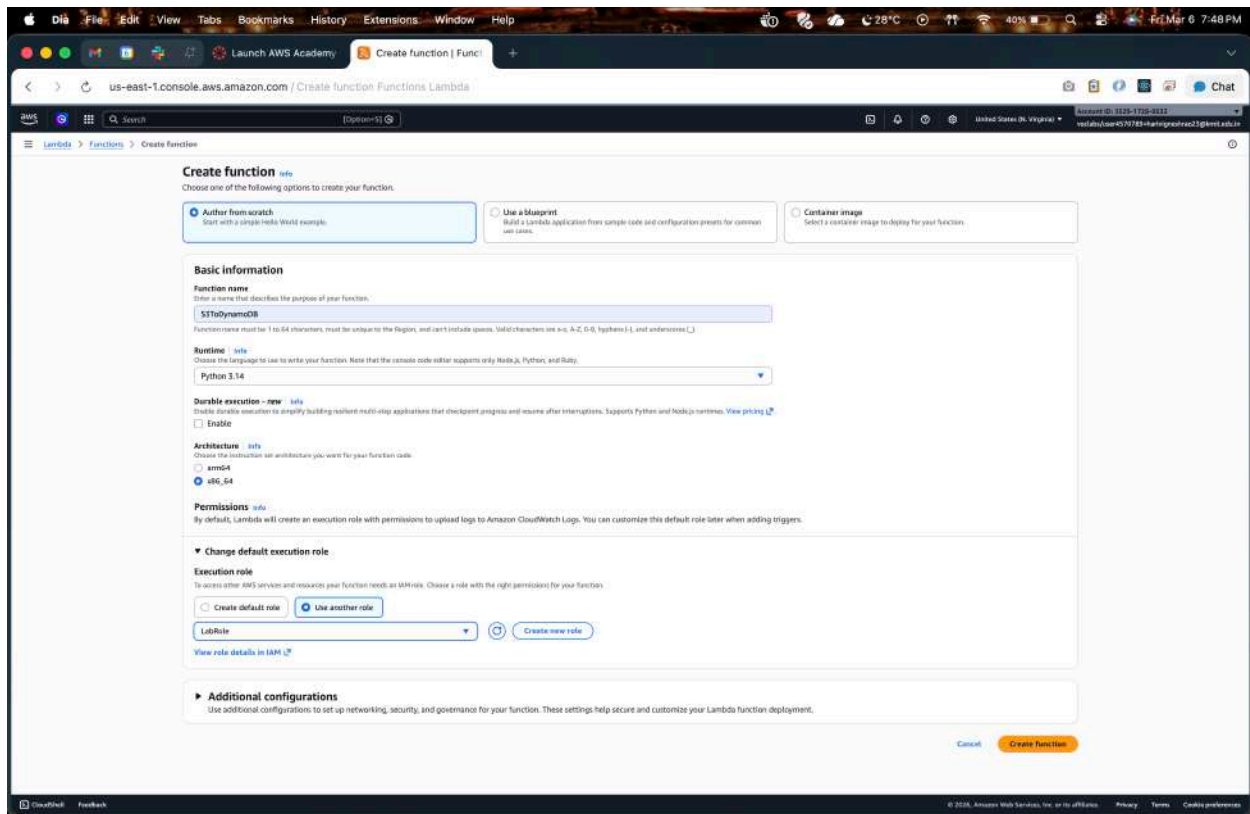


3. Open AWS Lambda Console: Navigate to the Lambda service by either searching for "Lambda" in the AWS Management Console or



4. Create a New Lambda Function:

- Click on the "Create function" button.
- Choose "Author from scratch".
- Enter a name for your function (e.g., "S3ToDynamoDB").
- Choose Python as the runtime.
- Under "Permissions", create a new role with basic Lambda permissions or choose an existing role that has permissions to access S3 and DynamoDB.
- Click on the "Create function" button.



5. Upload Code:

- Write the Python code for your Lambda function. This code will be triggered whenever an object is uploaded to the specified S3 bucket.

- Here's an example code snippet:

```
import boto3
```

```
from uuid import uuid4
```

```
def lambda_handler(event, context):
```

```
    dynamodb = boto3.resource('dynamodb')
```

```
    dynamoTable = dynamodb.Table('newtable2')
```

```
    if 'Records' in event:
```

```
        for record in event['Records']:
```

```
            bucket_name = record['s3']['bucket']['name']
```

```
            object_key = record['s3']['object']['key']
```

```
            size = record['s3']['object'].get('size', -1)
```

```
            event_name = record['eventName']
```

```
            event_time = record['eventTime']
```

```
            dynamoTable.put_item(
```

```
                Item={
```

```
                    'unique': str(uuid4()),
```

```
                    'Bucket': bucket_name,
```

```
                    'Object': object_key,
```

```

'Size': size,

'Event': event_name,

'EventTime': event_time

}

)

```

else:

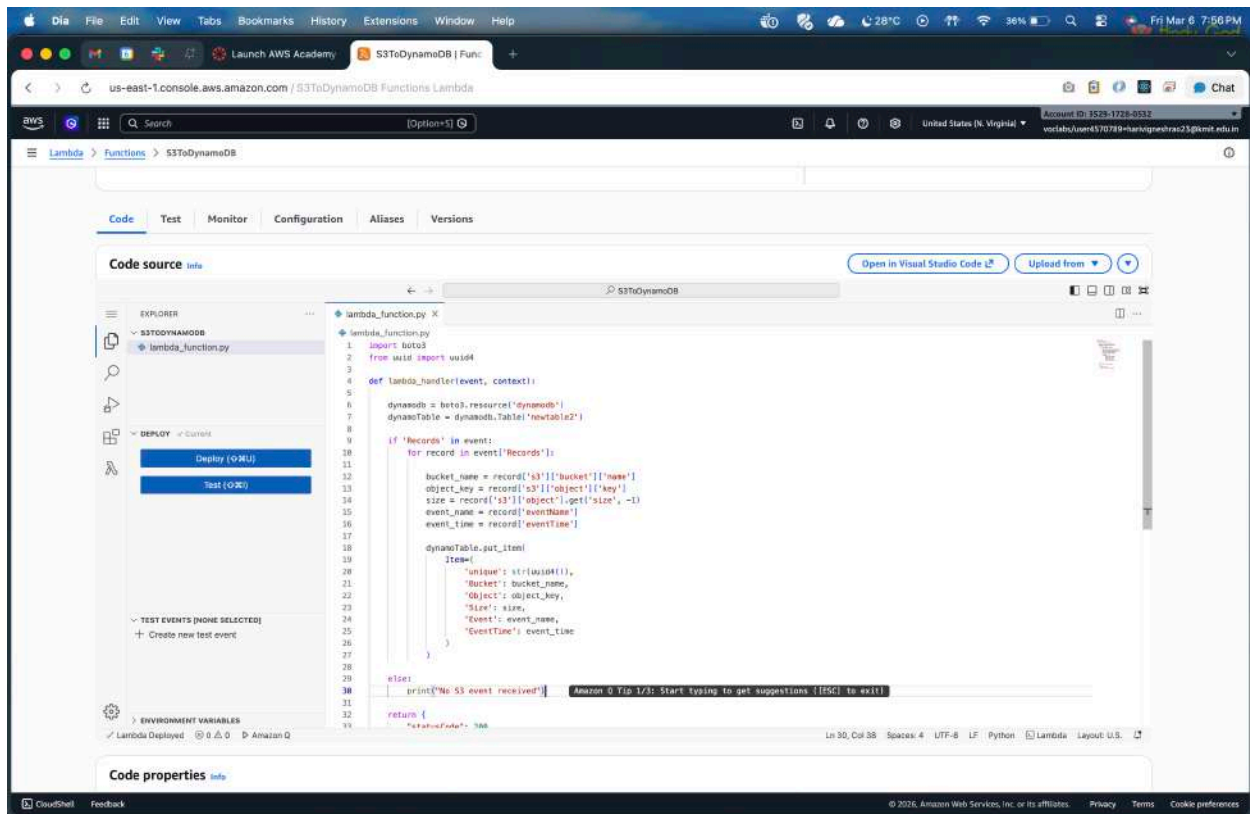
```
print("No S3 event received")
```

```
return {
```

```
    "statusCode": 200,
```

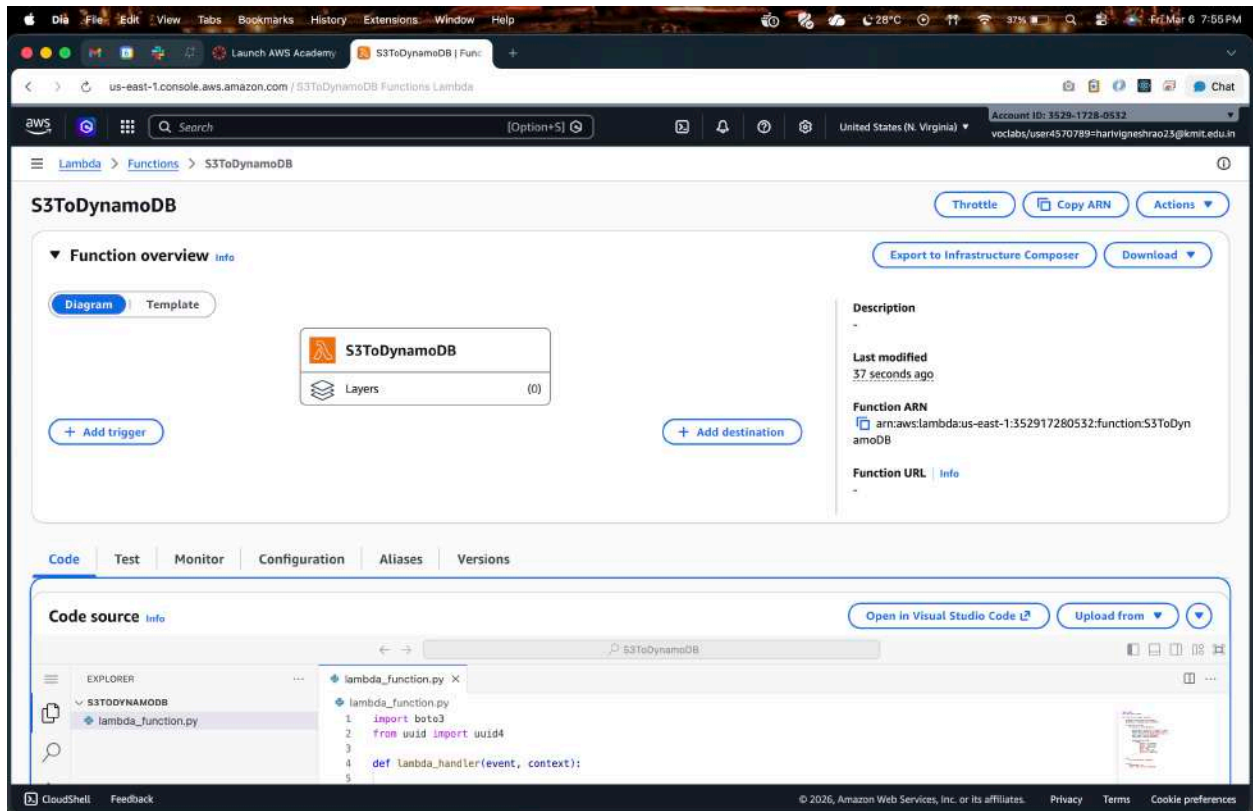
```
    "body": "Data stored in DynamoDB"
```

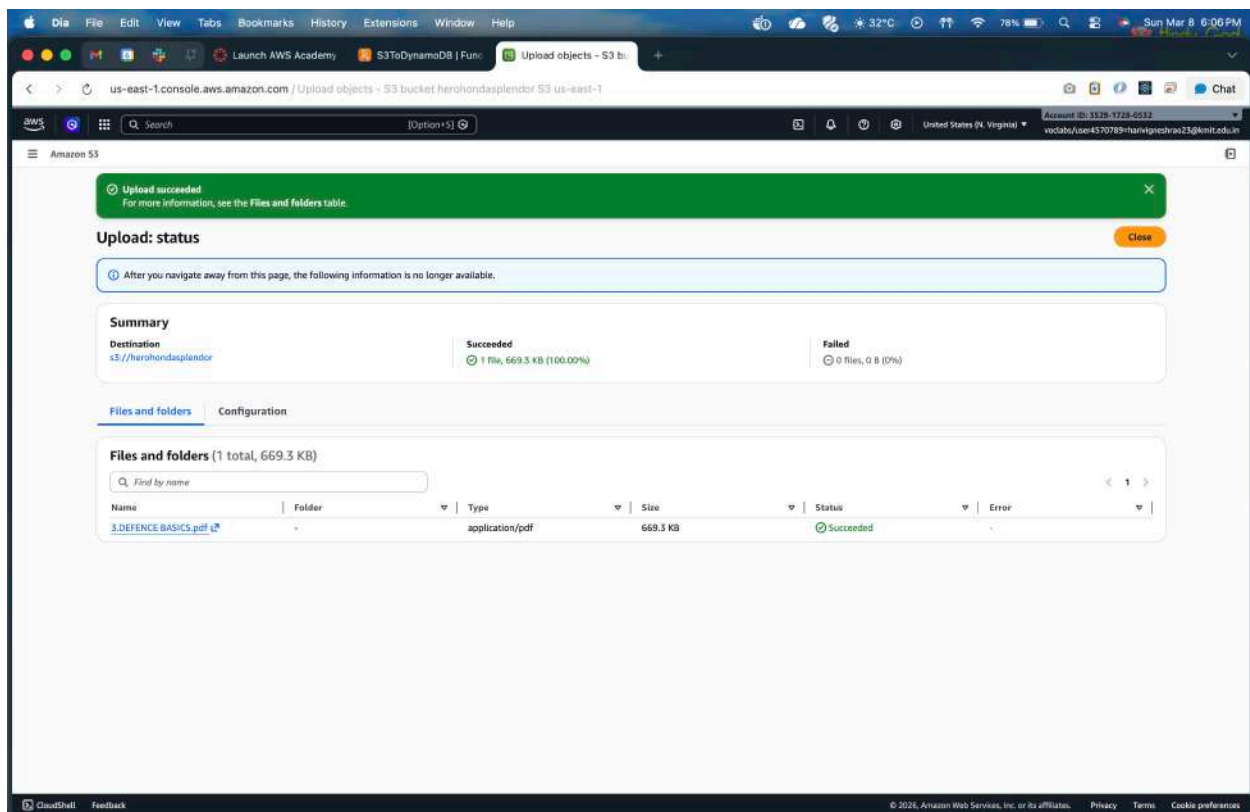
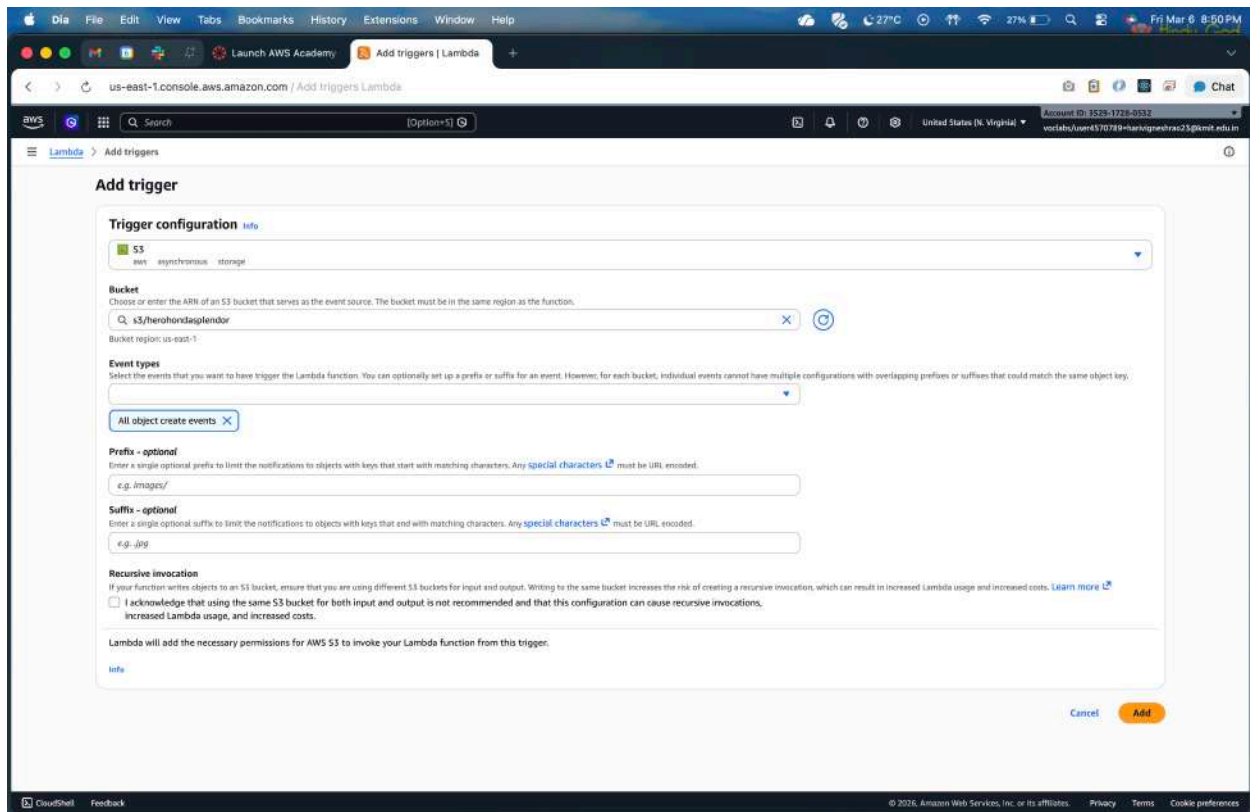
```
}
```

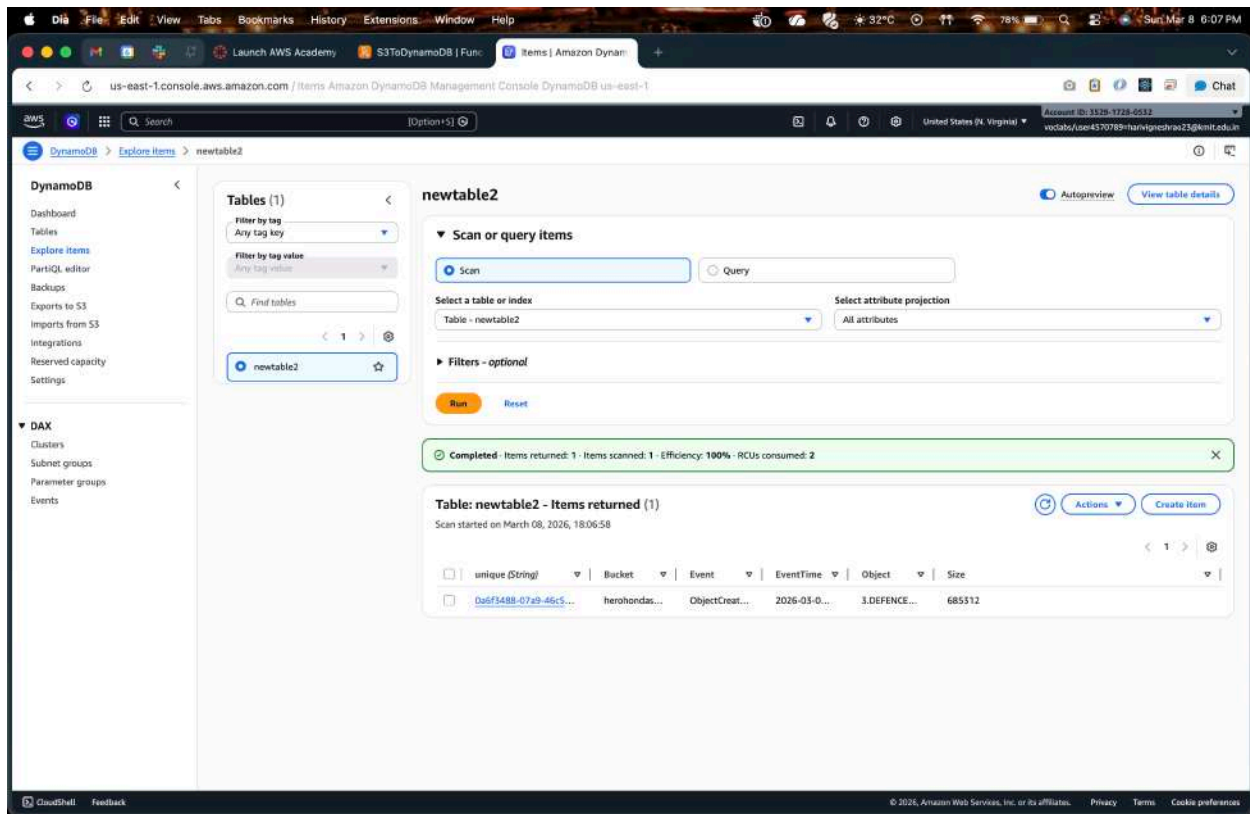


6. Configure Trigger:

- In the Designer section, click on "Add trigger".
- Select "S3" as the trigger type.
- Configure the trigger to listen to the S3 bucket where you want to trigger the Lambda function.
- Specify the event type (e.g., "All object create events").
- Click on the "Add" button.





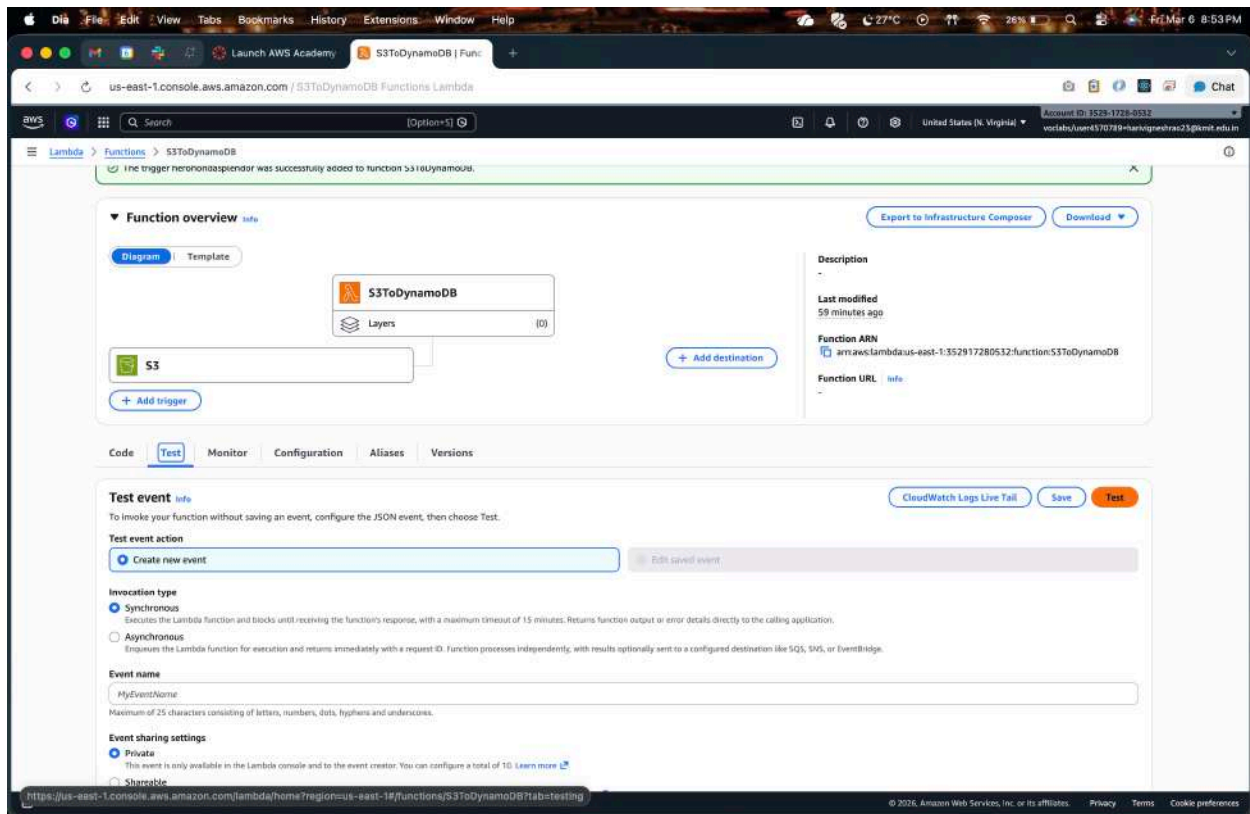


7. Configure Environment Variables (Optional):

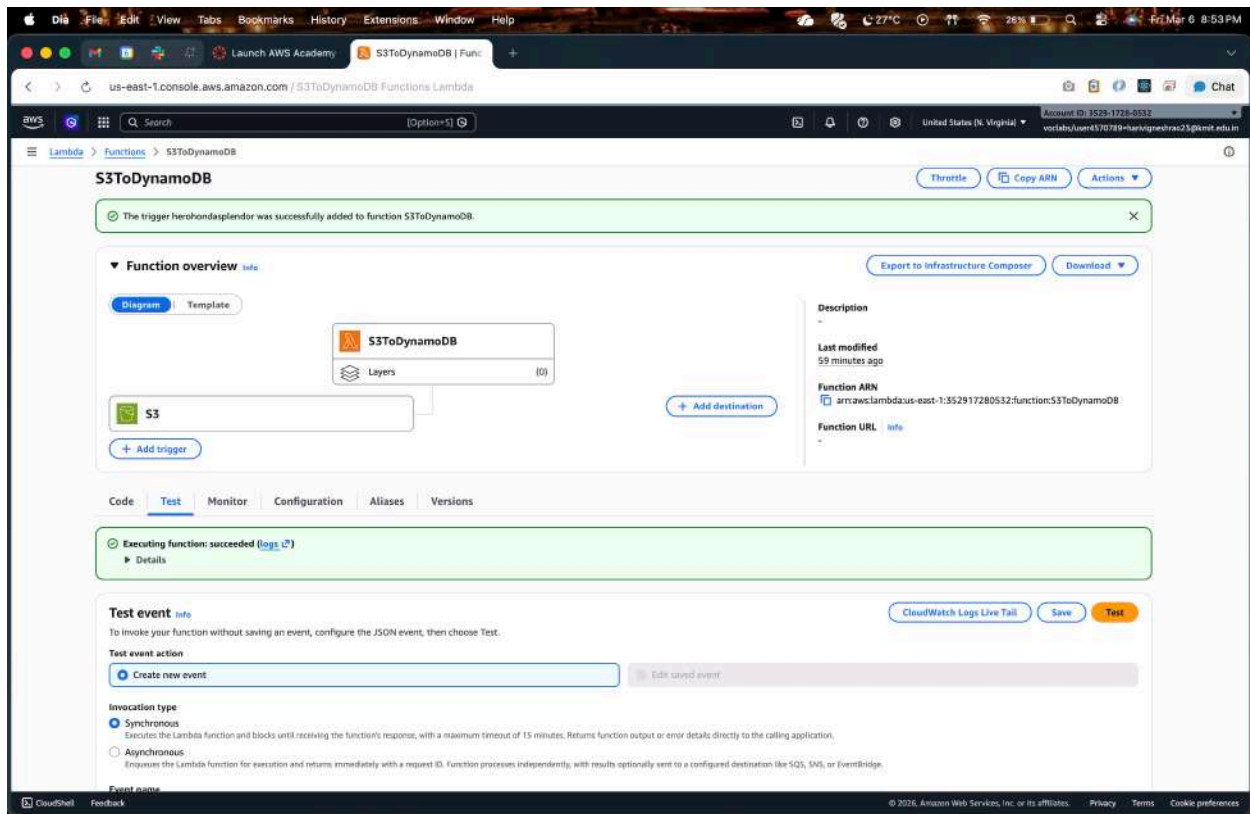
- If your Lambda function requires any environment variables (e.g., credentials for accessing DynamoDB), you can set them in the "Configuration" tab under "Environment variables".

8. Test Your Function:

- You can test your Lambda function manually by clicking on the "Test" button in the Lambda console.

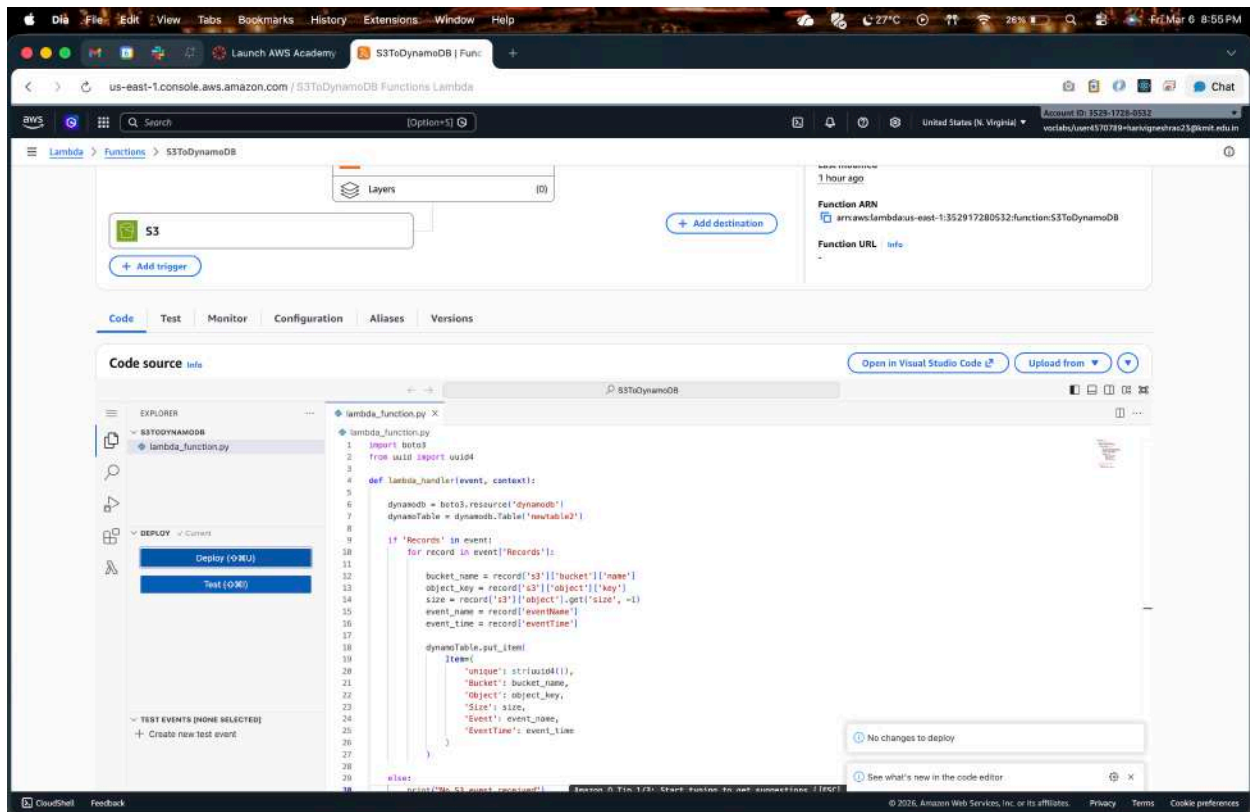


- You can create a sample test event or use a custom test event to simulate the S3 event that triggers your function.



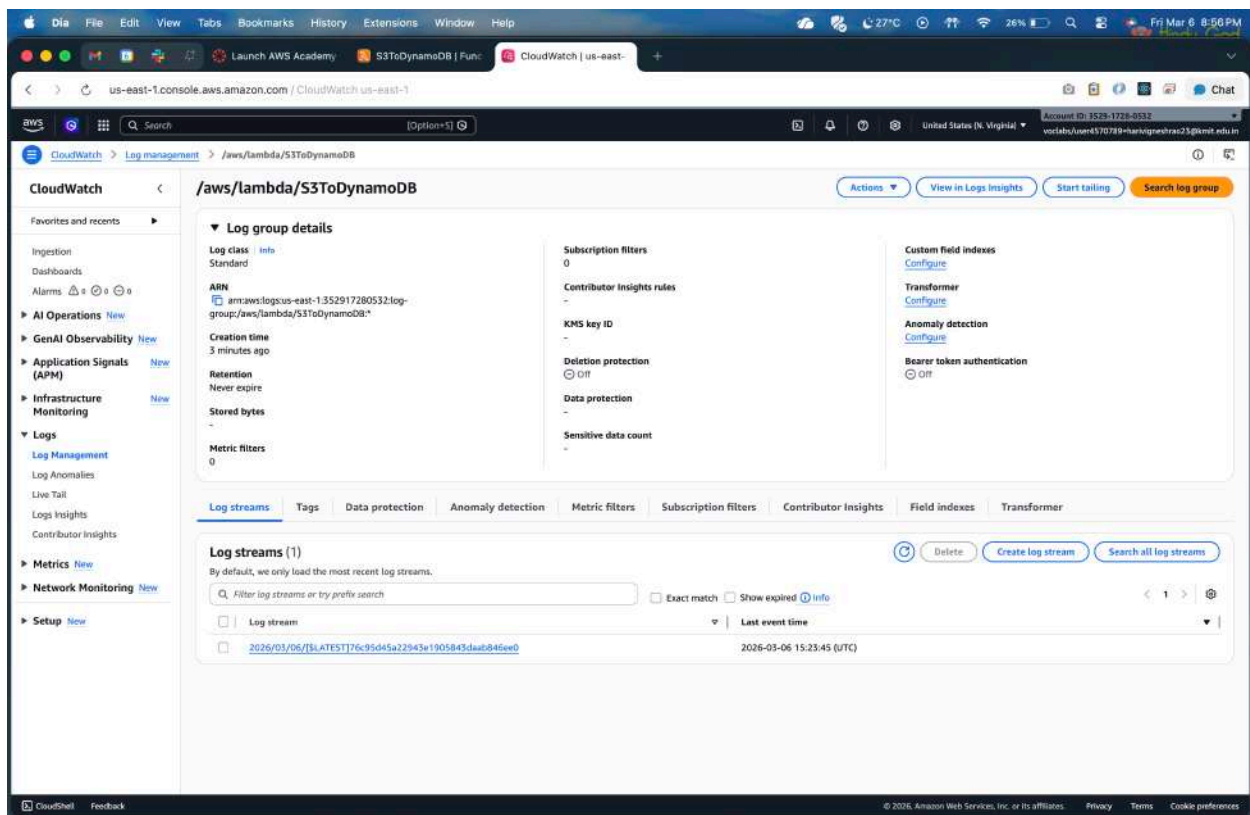
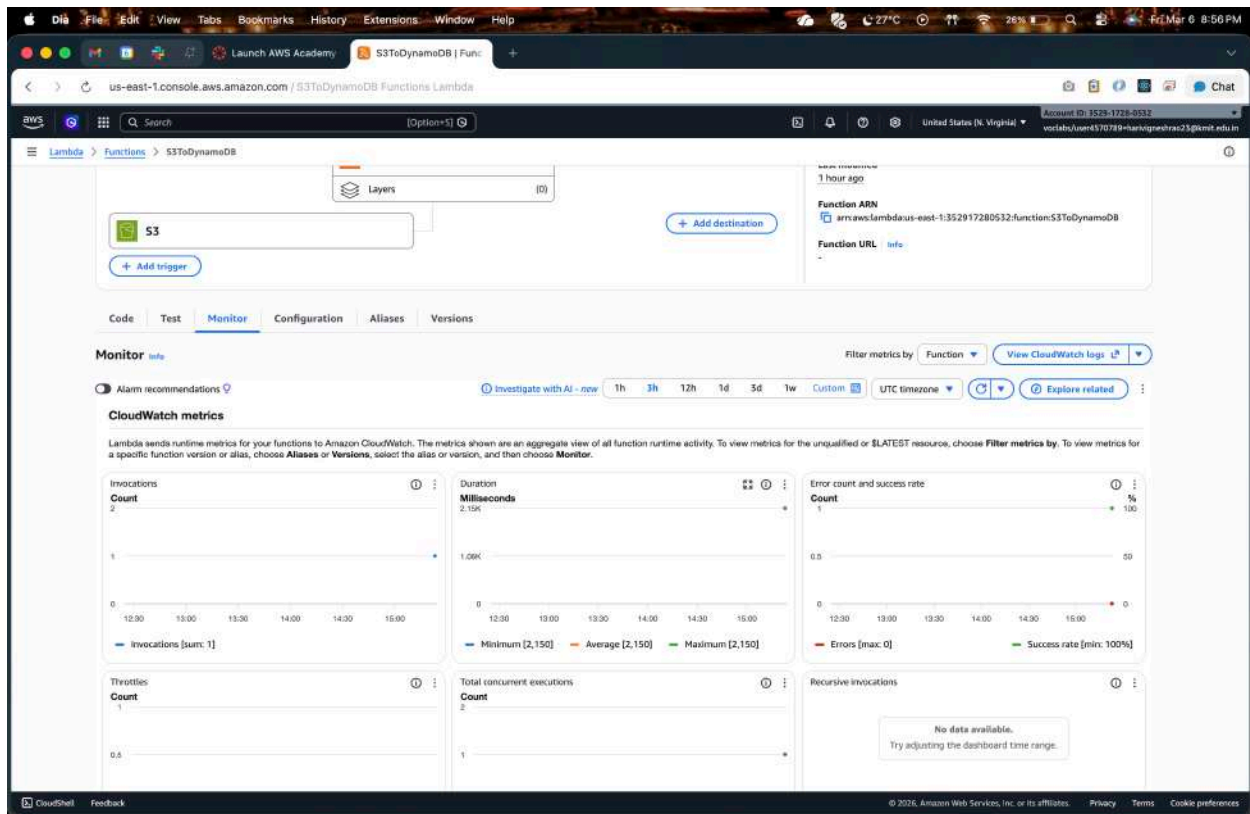
9. Save and Deploy:

- Click on the "Deploy" button to deploy your function.



10. Monitor Execution:

- You can monitor the execution of your Lambda function in the Monitoring tab of the Lambda console.
- You can also view logs to troubleshoot any issues that may arise during execution.



successfully set up a Lambda function to trigger whenever an object is uploaded to an S3 bucket and update a DynamoDB table accordingly.